Experiment No. 7

Title: Implementation of Topological Sorting

Batch: B3 **Roll No: 16014225801** **Experiment No.:7**

Aim: To study and implement Topological Sorting

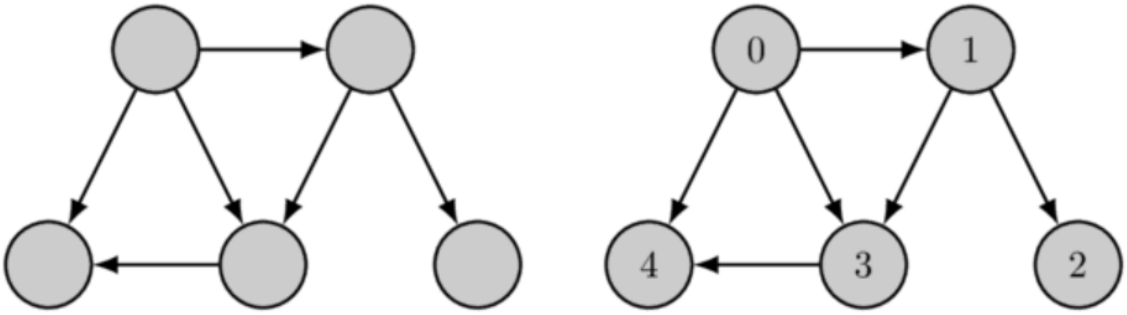
Resources needed: Text Editor, C/C++ IDE

Theory:

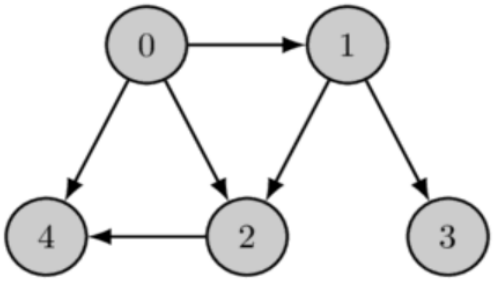
You are given a directed graph with n vertices and m edges. You have to find an **order of the vertices**, so that every edge leads from the vertex with a smaller index to a vertex with a larger one.

In other words, you want to find a permutation of the vertices (**topological order**) which corresponds to the order defined by all edges of the graph.

Here is one given graph together with its topological order:



Topological order can be **non-unique** (for example, if there exist three vertices a, b, c for which there exist paths from a to bb and from aa to cc but not paths from b to c or from c c to bb). The example graph also has multiple topological orders, a second topological order is the following:



A Topological order may **not exist** at all. It only exists, if the directed graph contains no cycles. Otherwise, there is a contradiction: if there is a cycle containing the vertices a and a and b, then a needs to have a smaller index than bb (since you can reach bb from aa) and also a bigger one (as you can reach aa from bb). The algorithm described in this article also shows by construction, that every acyclic directed graph contains at least one topological order.

A common problem in which topological sorting occurs is the following. There are nn variables with unknown values. For some variables, we know that one of them is less than the other. You

have to check whether these constraints are contradictory, and if not, output the variables in ascending order (if several answers are possible, output any of them). It is easy to notice that this is exactly the problem of finding the topological order of a graph with n vertices.

The Algorithm

To solve this problem, we will use depth-first search.

Let's assume that the graph is acyclic. What does the depth-first search do?

When starting from some vertex vv, DFS tries to traverse along all edges outgoing from vv. It stops at the edges for which the ends have been already been visited previously, and traverses along the rest of the edges and continues recursively at their ends.

Thus, by the time of the function call $dfs(v)$ has finished, all vertices that are reachable from v have been either directly (via one edge) or indirectly visited by the search.

Let's append the vertex v to a list, when we finish $dfs(v)$ Since all reachable vertices have already been visited, they will already be in the list when we append v Let's do this for every vertex in the graph, with one or multiple depth-first search runs. For every directed edge $v \to u$ in the graph, u will appear earlier in this list than v, because u is reachable from v. So if we just label the vertices in this list with $n-1, n-2, \ldots, 0$we have found a topological order of the graph. In other words, the list represents the reversed topological order.

These explanations can also be presented in terms of exit times of the DFS algorithm. The exit time for vertex v is the time at which the function call $dfs(v)$ finished (the times can be numbered from $0\,0$ to $n-1$). It is easy to understand that exit time of any vertex v is always greater than the exit time of any vertex reachable from it (since they were visited either before the call $dfs(v)$ or during it). Thus, the desired topological ordering are the vertices in descending order of their exit times.

Activity:

Write a program to implement the topological sorting for the following problem maintained on the hackerearth platform.

Solution:

Use case implementation

Code:

```
#include <stdio.h>
int tin[100];
int tout[100];
int timer = 0;
```

```c
void dfs(int node, int parent, int graph[100][100], int n)
{
    tin[node] = ++timer;

    for (int i = 1; i <= n; i++)
    {
        if (graph[node][i] && i != parent)
        {
            dfs(i, node, graph, n);
        }
    }

    tout[node] = ++timer;
}

int isAncestor(int x, int y)
{
    return tin[x] <= tin[y] && tout[y] <= tout[x];
}

int main()
{
    int graph[100][100] = {0};
    int n = 5;

    graph[1][2] = graph[2][1] = 1;
    graph[2][3] = graph[3][2] = 1;
    graph[2][4] = graph[4][2] = 1;
    graph[4][5] = graph[5][4] = 1;

    dfs(1, 0, graph, n);

    int x = 2;
    int y = 5;

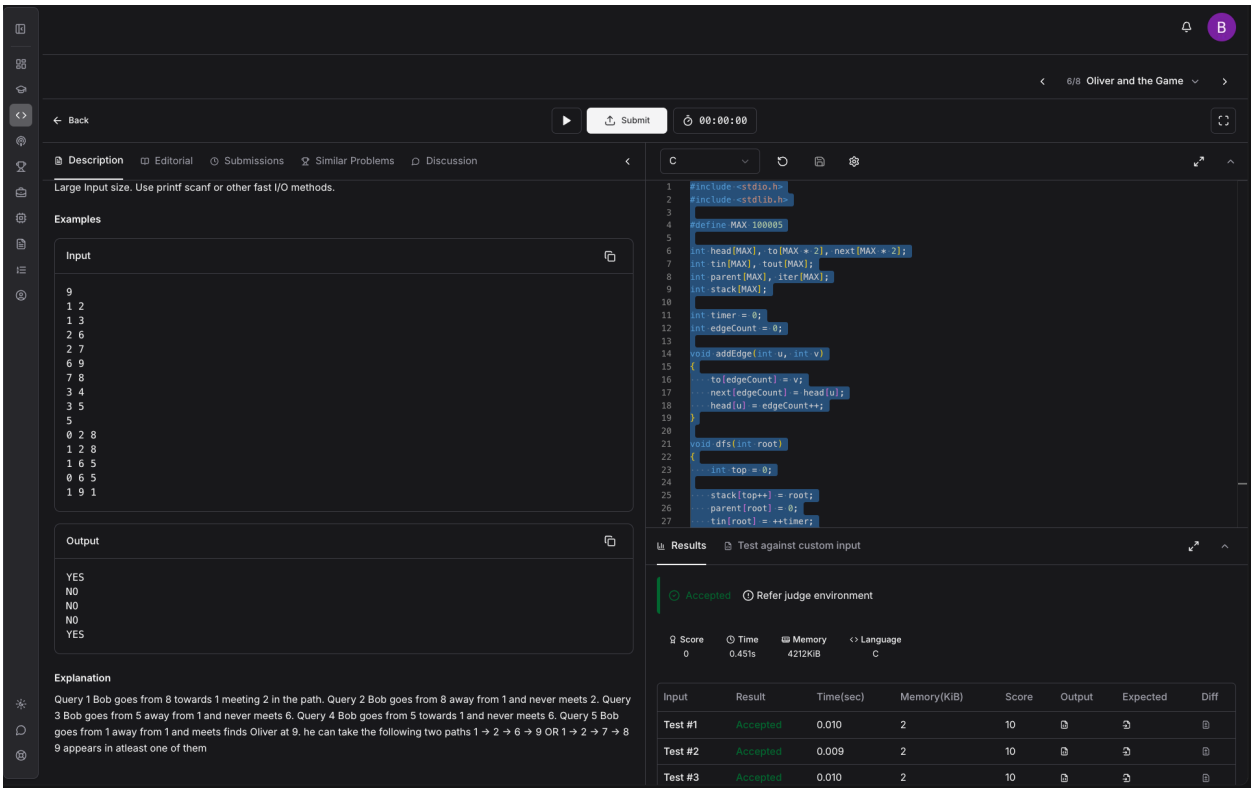
    if (isAncestor(x, y))
        printf("YES");
    else
        printf("NO");

    return 0;
}
```

Output:

```
(base) bhumit@bhumits-MacBook-Air exp7 % "/Users/bhumit/Desktop/Sem
5/cpl/code/exp7/exp7 usecase"
YES%
```

Hackerearth Problem



Conclusion:

Hence we performed the experiment.

References:

1. https://www.hackerearth.com/practice/algorithms/graphs/topological-sort/practice-problems/algorithm/oliver-and-the-game-3/
2. T.H. Coreman ,C.E. Leiserson,R.L. Rivest, and C. Stein, " Introduction to algorithms", 3rd Edition 2009, Prentice Hall India Publication
3. Antti Laaksonen, “Guide to Competitive Programming”,Springer,2018
4. https://drive.google.com/drive/folders/1c9mCFdPtIEW3TAs0tWpC9x2xJQwQ1E3j?usp=drive_link